

动态规划入门 区间类问题

主讲：黄凤林

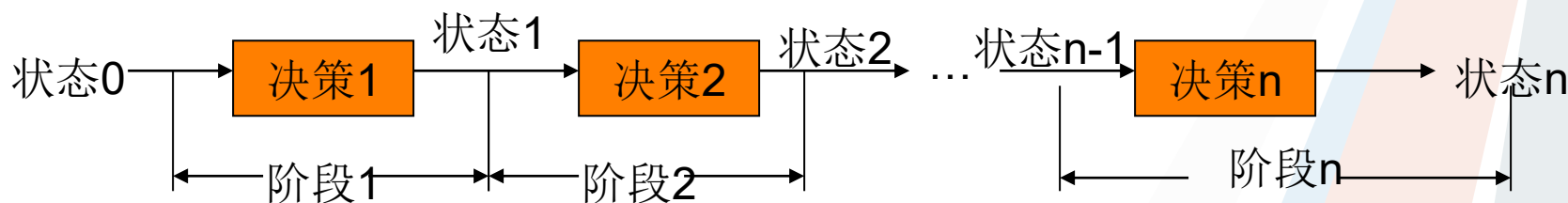


→ 动态规划

- **动态规划**(Dynamic Programming)是**研究多阶段决策过程最优化问题的一种数学方法**，英文缩写DP。
- 在动态规划中，**为了寻找一个问题的最优解（即最优决策过程），将整个问题划分成若干个相应的阶段，并在每个阶段都根据先前所作出的决策作出当前阶段最优决策，进而得出整个问题的最优解。**
- 能采用动态规划求解的问题的一般要具有三个性质：
- **最优化原理**：如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优化原理。
- **无后效性**：即某阶段状态一旦确定，就不受这个状态以后决策的影响。也就是说，某状态以后的过程不会影响以前的状态，只与当前状态有关。
- **有重叠子问题**：即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到。（该性质并不是动态规划适用的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势）

→ 动态规划

在现实生活中，有一类活动的过程，由于它的特殊性，可将过程分成若干个相互联系阶段，在它的每一个阶段都需要作出决策，从而使整个问题达到最好的活动效果。当然各个阶段决策的选取不是任何确定的，它依赖于当前面临的状态，又影响以后的发展，各个阶段决策确定后，就组成一个决策序列，因而也就确定了整个过程一条活动路线，这种把一个问题看作是一个前后关联具有链状结构的多阶段过程就称为多阶段决策过程，这种问题就称为多阶段决策问题。如下图：





→ 动态规划

- ◆ 动态规划是求解多阶段决策性最优解问题的有利武器，对思维难度要求高，常见实现策略**递推和记忆化搜索**两种，唯有多练习题目才能提高。
- ◆ **动态规划根据元素间关系，可以划分为常见模型：**
- ◆ 线性动态规划（一维、二维）
- ◆ 坐标动态规划（二维、三维）
- ◆ **区间动态规划（区间范围内）**
- ◆ 背包动态规划（9种背包）
- ◆ 树形动态规划（基于树上的记忆化搜索）
- ◆ 状压动态规划（数位、按位DP）
- ◆ 动态规划的优化（单调性、斜率、四边形）

→ 区间动态规划

- 所谓区间DP是**指在一段区间上(也即一个区间范围内)**进行的一系列动态规划(区间最优值)。
- 对于区间 DP 这一类问题，我们需要计算区间 $[1,n]$ 的答案，通常用一个**二维数组dp表示，其中 $dp[l][r]$ 表示区间 $[l,r]$ 。**
- 有些题目， **$dp[l][r]$ 由 $dp[l][r-1]$ 与 $dp[l+1][r]$ 推得；**
- 也有些题目，我们需要**枚举区间 $[l,r]$ 内的中间点，由两个子问题合并得到，也可以说 $dp[l][r]$ 由 $dp[l][k]$ 与 $dp[k+1][r]$ 推得，其中 $l \leq k < r$ 。**
- 对于长度为 n 的区间 DP，我们可以先计算 **$[1,1],[2,2] \dots [n,n]$ 的答案，再计算 $[1,2],[2,3] \dots [n-1,n]$ 以此类推，直到得到原问题的答案。**



区间动态规划

- 区间动态规划这种题目，在设置状态的时候，都可以设：
 $dp[i][j]$ ：为区间 $i \sim j$ 的最优值；
- 而 $dp[i][j]$ 的最优值，则由两个小区间合并而来的，为了划分这两个更小的区间，我们则需用用一个循环变量 k 来枚举，而一般的状态转移方程便是： $dp[i][j] = \max/\min\{dp[i][j], dp[i][k] + dp[k+1][j] + val\}, i \leq k < j$
- 具体情况则需要根据这个题目的实际含义进行变通即可
- 而区间dp的大致模板是：

```
for (int len=2;len<=n;len++){  
    for (int i=1;i+len-1<=n;i++){  
        int j=i+len-1;  
        for (int k=i;k<=j;k++)  
            dp[i][j]=max/min(dp[i][j],dp[i][k]+dp[k][j]+val);  
    }  
}
```

len枚举区间的长度

i和j分别是区间的起点和终点

k的作用是用来划分区间；时间复杂度为 n^3



→ 区间最大子段和(P1115)

题目描述

给出一段序列，选出其中连续且非空的一段使得这段和最大。

7

2 -4 3 -1 2 -4 3

dp[i]表示以i点结尾的最大值子段和

$dp[i] = dp[i-1] + a[i] \quad dp[i-1] > 0$

$Max\{dp[i]\}$

→ 区间求极值 (RMQ)

问题描述：给定 n 个数， m 个询问，针对每个询问求出第 i 个数到第 j 个数的最大值和最小值。

数据范围： $n \leq 100000, m \leq 1000000$ 。

样例输入

```
4 3
1 5 4 6
1 3
2 4
3 5
```

样例输出

```
5
6
6
```


➔ 问题分析

朴素1：针对每次询问，普通遍历 i 到 j 区间，求出最值，时间复杂度： $n*m$ 。（超时）

朴素2：对任意 i 到 j 位置DP预处理，用 $dp[i][j]$ 表示 i 到 j 中的最优值，则： $dp[i][j] = \max(dp[i][j-1], a[j])$ 然后针对每次查询， $O(1)$ 时间复杂度。

预处理时间复杂度和空间复杂度均为 $n*n$ 。

优化方法：怎么样对朴素2优化呢？

倍增思想（二分的反向）



→ RMQ

我们用 $dp[i][j]$ 来表示区间 $[i, i+2^j-1]$ 的最值，
采用分块思想，我们可以把区间 $[i, i+2^j-1]$ 平均分为两个区
间，因为 $j \geq 1$ 的时候该区间的长度始终为偶数，可以分为
区间 $[i, i+2^{(j-1)}-1]$ 和区间 $[i+2^{(j-1)}, i+2^j-1]$ ，即取两个
长度为 $2^{(j-1)}$ 的块取代和更新长度为 2^j 的块，那么最大值就
是这两个区间的最大值的最大值，转移方程为：

$$dp[i][j] = \min(dp[i][j-1], dp[i+2^{(j-1)}][j-1])$$

$$dp[i][j] = \max(dp[i][j-1], dp[i+2^{(j-1)}][j-1])$$

这样子的时间复杂度： $n \cdot \log n$ ，空间复杂度 $n \cdot \log n$
但是这样子预处理？如何查询答案呢



RMQ询问查询

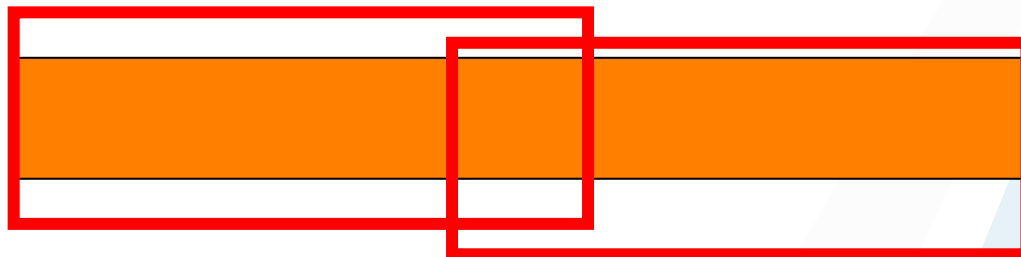
假设要查询从 m 到 n 这一段的最小值, 那么我们先求出一个最大的 k , 使得 k 满足 $2^k < (n - m + 1)$. 于是我们就可以把 $[m, n]$ 分成两个(部分重叠的)长度为 2^k 的区间: $[m, m + 2^k - 1]$, $[n - 2^k + 1, n]$;

而我们之前已经求出了 $dp[m][k]$ 为 $[m, m + 2^k - 1]$ 的最小值, $dp[n - 2^k + 1][k]$ 为 $[n - 2^k + 1, n]$ 的最小值

我们只要返回其中更小的那个, 就是我们想要的答案, 这个算法的时间复杂度是 $O(1)$ 的.

例如, $rmq(1, 11) = \min(dp[1][3], dp[3][3])$

由此我们要注意的是预处理 $dp[i][j]$ 中的 j 值只需要计算 $\log(n+1)/\log(2)$ 即可, 而 i 值我们也只需要计算到 $n - 2^k + 1$ 即可。





➔ 练习题目

例子: Poj 3264,

找出任意给定区间中, 最大值和最小值之间的差

luogu:[P1816 忠诚](#)

Luogu:P2251

P1440: 求m区间内的最小值



石子合并

➤ 题目描述

在操场上沿一直线排列着 n 堆石子。现要将石子有次序地合并成一堆。

规定每次只能选相邻的两堆石子合并成新的一堆，并将新的一堆石子数计为该次合并的得分。我们希望这 $n-1$ 次合并后得到的得分总和最小。

➤ 数据范围： $n \leq 300$



➔ 石子合并---问题分析

➤ **贪心法：**每次选取相邻两个最小值进行合并

N=4 石子数分别为4 2 3 4。

用贪心法的合并过程如下：

第一次 4 2 3 4 得分 5

第二次 4 5 4 得分9

第三次 9 4 得分13

总分：27分

然而有更好的方案：

第一次4 2 3 4 得分 6

第二次6 3 4 得分7

第三次6 7得分13

总分：26

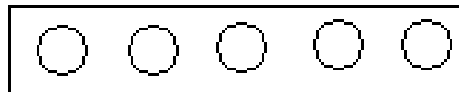
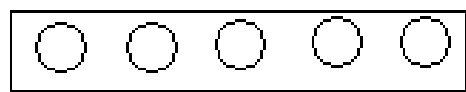
显然，贪心法是错误的。

石子合并---问题分析

- ✧ 假设只有2堆石子，显然只有1种合并方案
- ✧ 如果有3堆石子，则有2种合并方案， $((1,2),3)$ 和 $(1,(2,3))$
- ✧ 如果有k堆石子呢？
- ✧ 不管怎么合并，总之最后总会归结为2堆，如果我们将最后两堆分开，左边和右边无论怎么合并，都必须满足最优合并方案，整个问题才能得到最优解。如下图：

左边合并成一堆

右边合并成一堆



最后合并成一堆

这样就是把一个长度为 $1 \sim n$ 的区间，最后一次，枚举中间位置，分成两部分的问题了！

石子合并---问题分析

- ✧ 因此设 $dp[i][j]$ 表示 $i \sim j$ 区间的石子合并成一堆的最优方案即最小花费，假设在我们进行 i 和 j 的状态转移的时候必然更小的区间已经求出了最优的方案数，那么我们只需要用 k 去划分石子数去进行状态转移即可。
- ✧ 以得到状态转移方程：
- ✧ $dp[i][j] = \min(dp[i][j], dp[i][k] + dp[k+1][j] + s[i][j])$
- ✧ $s[i][j]$ 表示从第 i 堆到第 j 堆加在一起的石头数量
- ✧ 整体时间复杂度： n^4 ，但是 $s[i][j]$ 我们可以用前缀和预处理优化，则整体时间复杂度为 N^3 。

➔ 石子合并---参考代码

- ✧ 在代码实现上，我们需要注意
- ✧ 区间长度应从2开始的，诸如 $dp[1][1]$, $dp[2][2]$ 都是不需要花费价值的，即初始值0；
- ✧ 状态转移方程是 $dp[i][k] + dp[k+1][j] + s[i][j])$ 而不是 $dp[i][k] + dp[k][j] + s[i][j])$ 合并的是两堆不同的石子而不是一堆

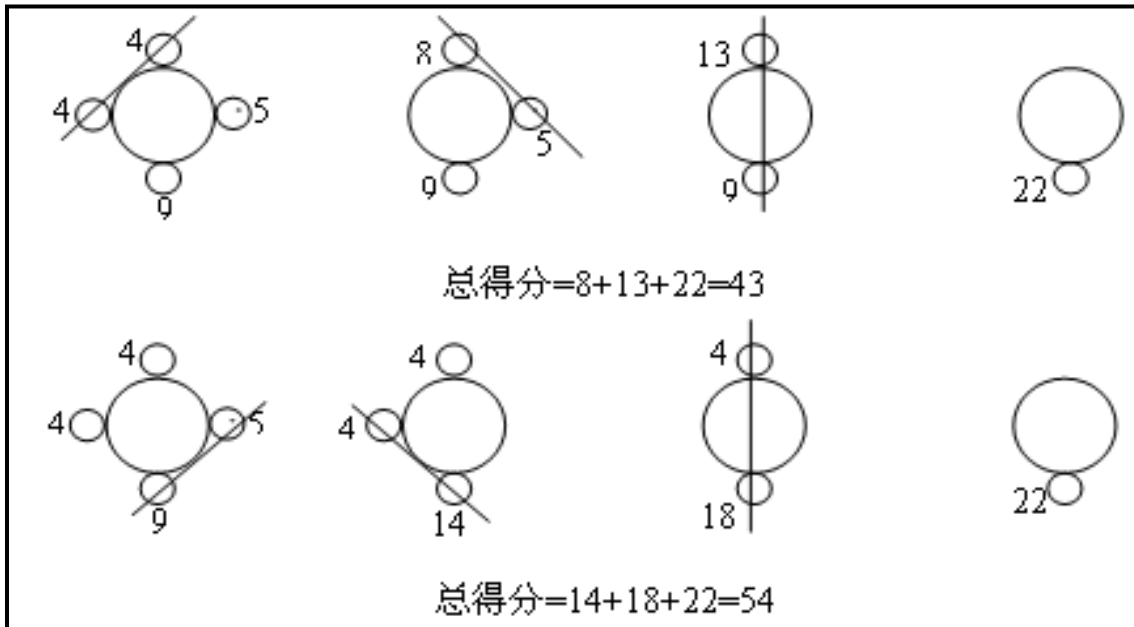
```
for (int i=1;i<=n;i++){ cin>>a[i];sum[i]=sum[i-1]+a[i];}
for (int i=1;i<=n;i++)
    for (int j=i;j<=n;j++) cost[i][j]=sum[j]-sum[i-1];
for (int len=2;len<=n;len++){
    for (int i=1;i<=n;i++)
    {
        int j=i+len-1;
        if (j>n) continue;
        for (int k=i;k<j;k++){
            if ((dp[i][k]+dp[k+1][j]+s[i][j]<dp[i][j]))
                d[i][j]=d[i][k]+d[k+1][j]+s[i][j];
        }
    }
}
} printf("%d\n",dp[1][n]);|
```



问题扩展

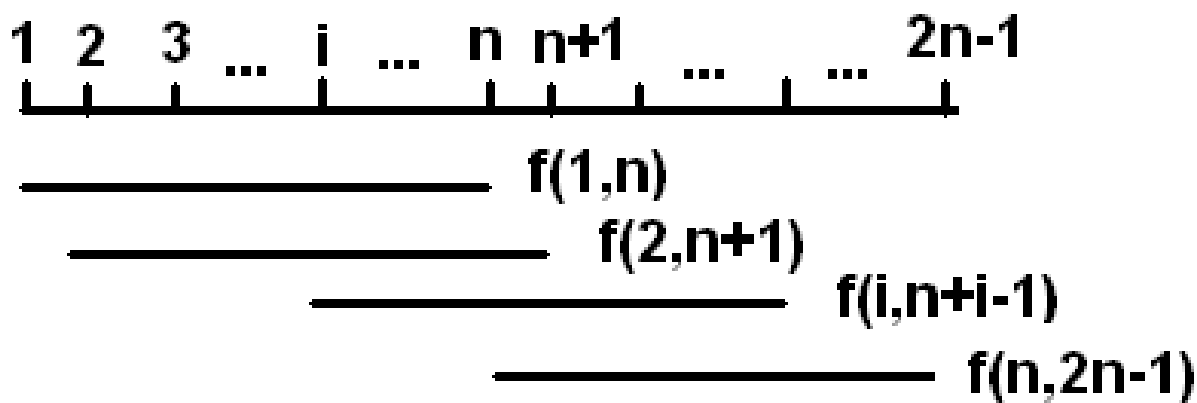
✧ 如何给定的序列不是一条直线，而是一个环呢？

✧ 例如：



问题扩展---优化

- ✧ 由于石子堆是一个圈，因此我们可以枚举分开的位置，首先将这个圈转化为链，因此总的时间复杂度为 $O(n^4)$ 。
- ✧ 这样显然很高，其实我们可以将这条链延长2倍，扩展成 $2n-1$ 堆，其中第1堆与 $n+1$ 堆完全相同，第 i 堆与 $n+i$ 堆完全相同，这样我们只要对这 $2n$ 堆动态规划后，枚举 $dp(1,n), dp(2,n+1), \dots, dp(n, 2n-1)$ 取最优值即可即可。
- ✧ 时间复杂度为 $O(8n^3)$,如下图：





➔ 能量项链

➤ 问题描述【NOIP2006提高组】

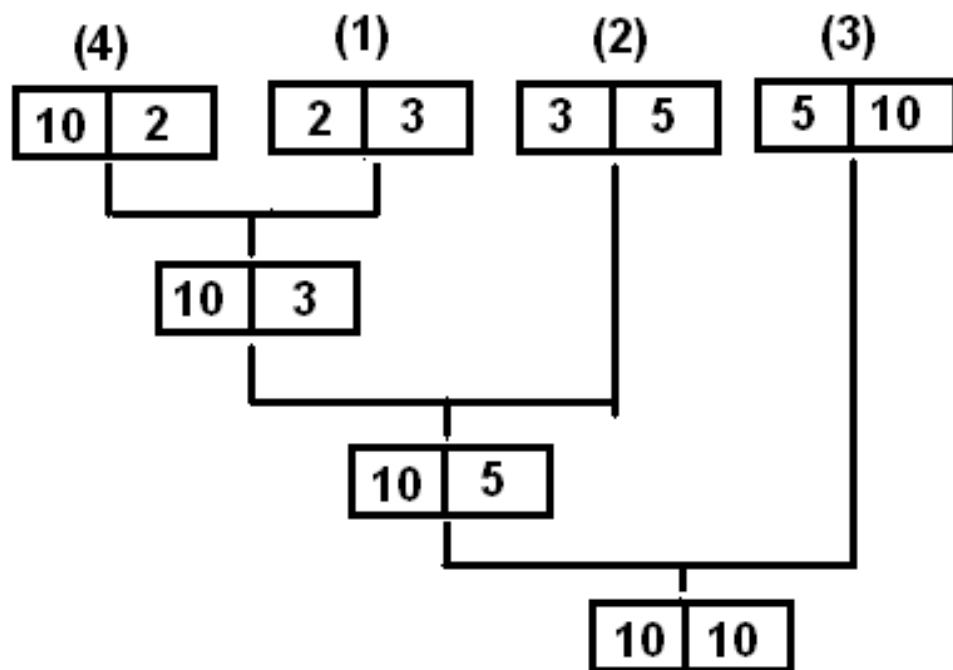
在Mars星球上，每个Mars人都随身佩带着一串能量项链。在项链上有N颗能量珠。能量珠是一颗有头标记与尾标记的珠子，这些标记对应着某个正整数。

对于相邻的两颗珠子，前一颗珠子的尾标记一定等于后一颗珠子的头标记。如果前一颗能量珠的头标记为m，尾标记为r，后一颗能量珠的头标记为r，尾标记为n，则聚合后释放的能量为 $m \times r \times n$ （Mars单位），新产生的珠子的头标记为m，尾标记为n。

显然，对于一串项链不同的聚合顺序得到的总能量是不同的，请设计一个聚合顺序，使一串项链释放出的总能量最大

问题分析

- ✧ 假设 $N=4$ ，4颗珠子的头标记与尾标记依次为
(2, 3) (3, 5) (5, 10) (10, 2)。
- ✧ 我们用记号 $\oplus$ 表示两颗珠子的聚合操作，释放总能量：
 $((4 \oplus 1) \oplus 2) \oplus 3) = 10 \times 2 \times 3 + 10 \times 3 \times 5 + 10 \times 5 \times 10 = 710$





➔ 问题分析

- ✧ 该题做法与石子合并完全类似。
- ✧ 设链中的第 i 颗珠子头尾标记为 $(S_{i-1}$ 与 $S_i)$ 。
- ✧ 令 $dp[i][j]$ 表示从第 i 颗珠子一直合并到第 j 颗珠子所能产生的最大能量，则有：
- ✧ $dp[i][j] = \text{Max}\{dp[i][k] + dp[k+1][j] + S_{i-1} * S_k * S_j, i \leq k < j\}$
- ✧ 边界条件： $dp[i][j] = 0, 1 \leq i < k < j \leq n$
- ✧ 至于圈的处理，与石子合并方法完全相同，时间复杂度 $O(8n^3)$ 。



→ 矩阵连乘

➤ 题目描述

给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，考察这 n 个矩阵的连乘积 $A_1 A_2 \dots A_n$ ，由于矩阵乘法满足结合律，故计算矩阵的连乘积可以有許多不同的计算次序，这种计算次序可以用加括号的方式来确定。

矩阵连乘积的计算次序与其计算量有密切关系。例如，考察计算3个矩阵 $\{A_1, A_2, A_3\}$ 连乘积的例子。

设这3个矩阵的维数分别为 10×100 ， 100×5 ，和 5×50

若按 $(A_1 A_2) A_3$ 计算，3个矩阵连乘积需要的数乘次数为：

$$10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$$

若按 $A_1 (A_2 A_3)$ 计算，则总共需要：

$$100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000 \text{ 次数乘。}$$

现在你的任务是给出一个矩阵连乘式，计算其需要的最少乘法次数。



➔ 整数划分

➤ 问题描述【落谷P1018】

给出一个长度为 n 的数

要在其中加 $m-1$ 个乘号，分成 m 段

使这 m 段的乘积之积最大

➤ 数据范围： $m < n \leq 20$

➤ 有 T 组数据， $T \leq 10000$



➔ 问题分析

✧ **贪心法：** 尽可能平均分配各段，这样最终的值将尽可能大。

✧ 但有反例。如191919分成3段

$$19*19*19=6859$$

但 $191*91*9=156429$ ，显然乘积更大。

✧ **将一个数分成若干段乘积后比该数小**，因为输入数不超过20位，因此不需高精度运算。

✧ 证明：

✧ 假设AB分成A和B,且 $A, B < 10$,则有

✧ $AB = 10*A + B > A*B$ (相当于B个A相加)

✧ 同理可证明A,B为任意位也成立



➔ 问题分析

- ✧ 具体做法，与护卫队那种题目很类似，我们用：
- ✧ $dp[i][j]$ 表示把这个数前 i 位分成 j 段得到的最大乘积，则：
- ✧ $dp[i][j] = dp[k][j-1] * A[k+1, i]$, $1 < k < i \leq n$, $j \leq m$
- ✧ 时间复杂度为 $O[m^2n]$
- ✧ 由于有10000组数据，因此估计时间复杂度为 $10000 * 20^3 = 8 * 10^7$
- ✧ 至于说输出，记录转移的父亲就可以了。



➔ 关灯

➤ 题目描述

宁智贤得到了一份有趣而高薪的工作。每天早晨她必须关掉她所在村庄的街灯，所有的街灯都被设置在一条直路的同一侧。

宁智贤每晚到早晨5点钟都在晚会上，然后她开始关灯。开始时，她站在某一盏路灯的旁边。每盏灯都有一个给定功率的电灯泡，因为宁智贤有着自觉的节能意识，她希望在耗能总数最少的情况下将所有的灯关掉。

宁智贤因为太累了，所以只能以 1m/s 的速度行走。关灯不需要花费额外的时间，因为当她通过时就能将灯关掉。

编写程序，计算在给定路灯设置，灯泡功率以及宁智贤的起始位置的情况下关掉所有的灯需耗费的最小能量。



→ 关灯

✧ 输入格式

第一行包含一个整数 N , $2 \leq N \leq 1000$, 表示该村庄路灯的数量。

第二行包含一个整数 V , $1 \leq V \leq N$, 表示宁智贤开始关灯的路灯号码。

接下来的 N 行中, 每行包含两个用空格隔开的整数 D 和 W , 用来描述每盏灯的参数, 其中 $0 \leq D \leq 1000$, $0 \leq W \leq 1000$ 。 D 表示该路灯与村庄开始处的距离(用米为单位来表示), W 表示灯泡的功率, 即在每秒种该灯泡所消耗的能量数。路灯是按顺序给定的。

✧ 输出格式

第一行即唯一的一行应包含一个整数, 即消耗能量之和的最小值。

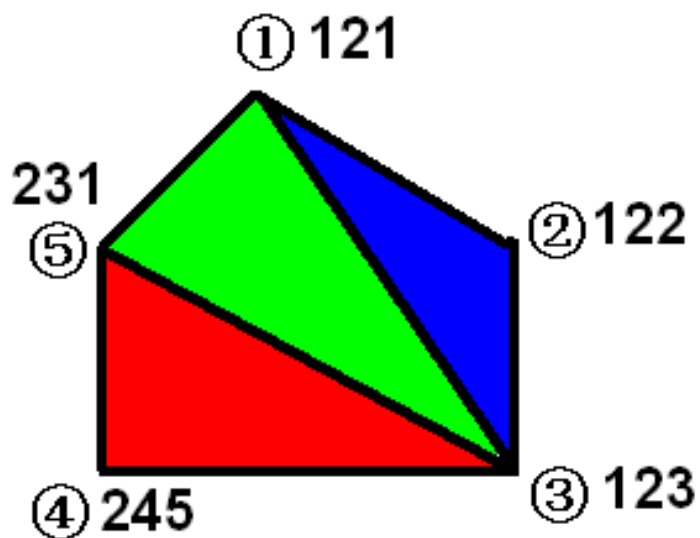


问题分析

- ✧ 根据分析，对于某一段路灯区间 $[i,j]$ ，我们必然要在关完灯后站在 i 或 j 上，否则就会多走路。所以它必然由 $[i+1,j]$ 或 $[i,j-1]$ 得来。
- ✧ 进一步思考，最后站在 i 或 j 上，对于向更大区间的转移来说是两种不同的情况，所以我们增设一维。
- ✧ 设 $f[i][j][0]$ 表示关完 $[i,j]$ 后站在左边的 i 上；
- ✧ 设 $f[i][j][1]$ 表示关完 $[i,j]$ 后站在右边的 j 上。可得状态转移方程：
 $f[i][j][0] = \min\{f[i+1][j][0] + \text{从 } i+1 \text{ 走到 } i \text{ 的时间里未关的灯的能耗}, f[i+1][j][1] + \text{从 } j \text{ 走到 } i \text{ 的时间里未关的灯的能耗}\}$
- ✧ 一开始没想通为什么还要从 $[i+1][j][1]$ 转移，我觉得从 j 走到 i 会比另一种情况多走，但实际上最终站在左边或右边是两种完全不同的走法，转移方程中后者完全有可能通过玄妙的走位以很少的时间关完灯站好，所以即使从右边走到 i 比较远，后者也有可能大于前者。至于某段时间里未关的灯的能耗怎么求呢？
- ✧ 我们预处理一个前缀和数组 $s[i]$ ，表示 $1 \sim i$ 路灯单位时间的能耗总和，从而求得数组 $cost[i][j]$ ，表示除了 $[i,j]$ 外的灯单位时间的能耗总和 $cost[i][j] = s[n] - s[j] + s[i-1]$ ；那么，从 $i+1$ 走到 i 的时间里未关的灯的能耗即为 $cost[i][i+1] * (dis[i+1] - dis[i])$ 。
- ✧ 同理可得 $f[i][j][1]$ 的转移方程。

→ 凸多边形的三角剖分

- ✧ 给定由N顶点组成的凸多边形，每个顶点具有权值
- ✧ 将凸N边形剖分成N-2个三角形
- ✧ 求N-2个三角形顶点权值乘积之和最小？



上述凸五边形分成 $\triangle 123, \triangle 135, \triangle 345$
 三角形顶点权值乘积之和为：
 $121 * 122 * 123 + 121 * 123 * 231 + 123 * 245 * 231 = 12214884$



→ 问题分析

- 性质：一个凸多边形剖分一个三角形后，可以将凸多边形剖分成三个部分：
- ✧ 一个三角形
 - ✧ 二个凸多边形（图2可以看成另一个凸多边形为0）

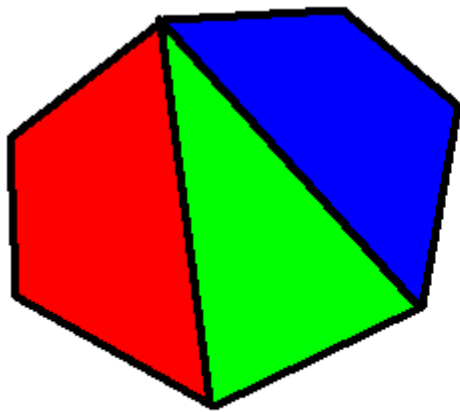


图1

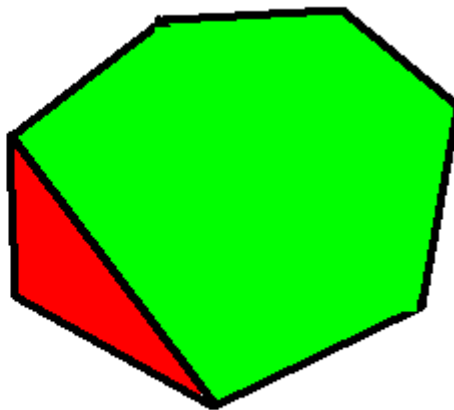
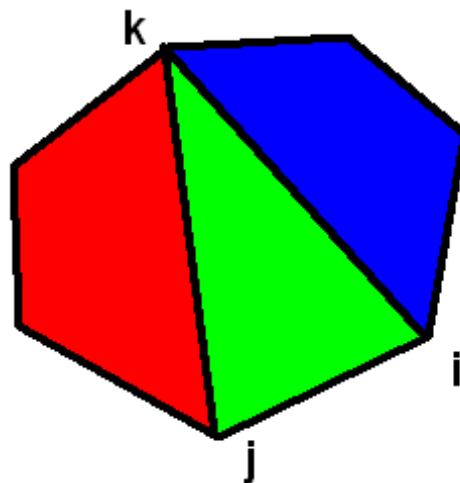


图2



→ 动态规划

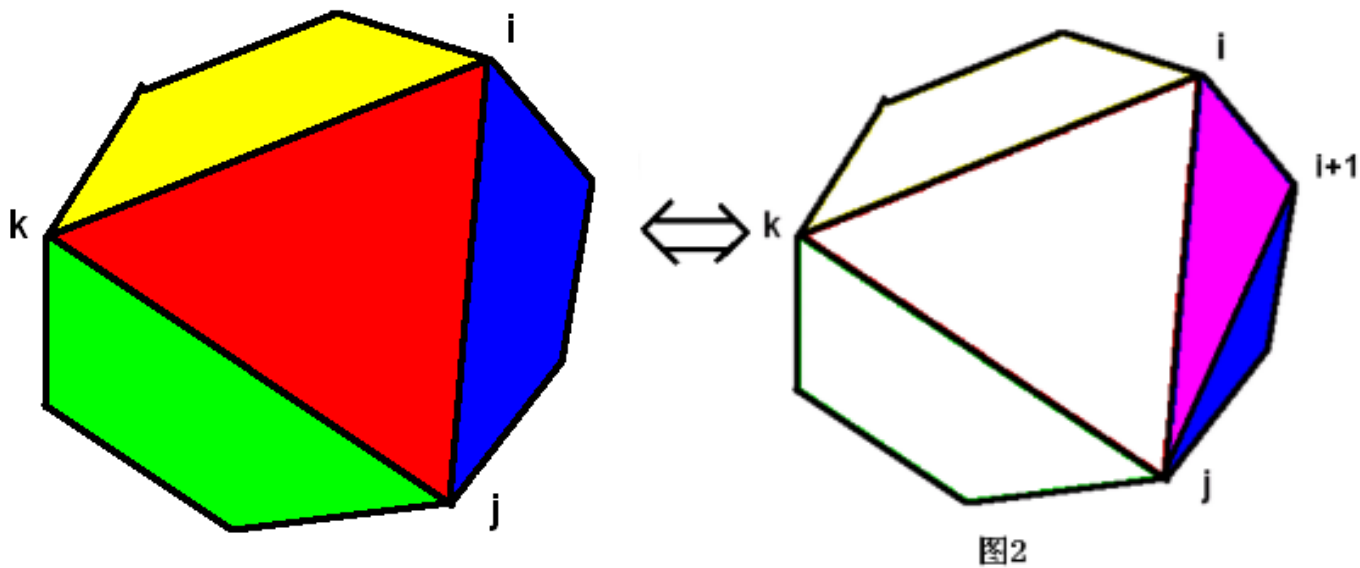
- ✧ 如果我们按顺时针将顶点编号，则可以相邻两个顶点描述一个凸多边形。
- ✧ 设 $dp[i][j]$ 表示 $i \sim j$ 这一段连续顶点的多边形划分后最小乘积
- ✧ 枚举点 k ， i 、 j 和 k 相连成基本三角形，并把原多边形划分成两个子多边形，则有
- ✧ $dp[i][j] = \min\{dp[i][k] + dp[k][j] + a[i] * a[j] * a[k]\}$
- ✧ $1 \leq i < k < j \leq n$
- ✧ 时间复杂度 $O(n^3)$





→ 讨论

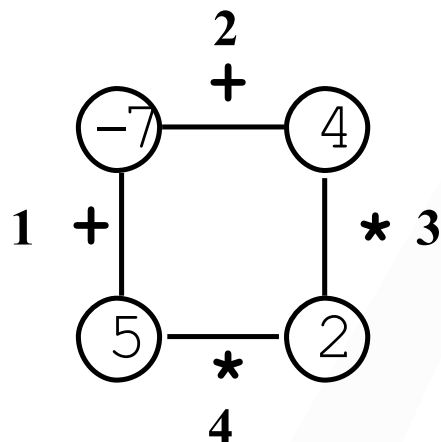
➤ 为什么可以不考虑这种情况？



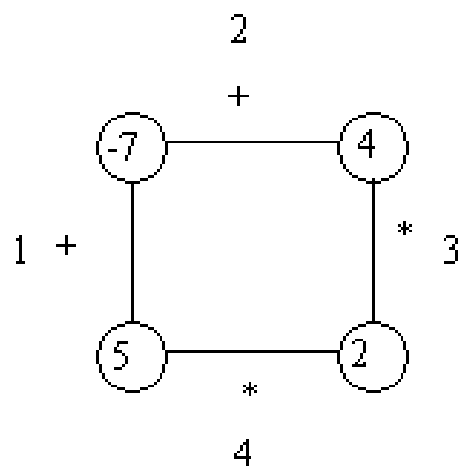
可以看出图1和图2是等价的，也就是说如果存在图1的剖分方案，则可以转化成图2的剖分方案，因此可以不考虑图1的这种情形。

→ 多边形 (IOI98)

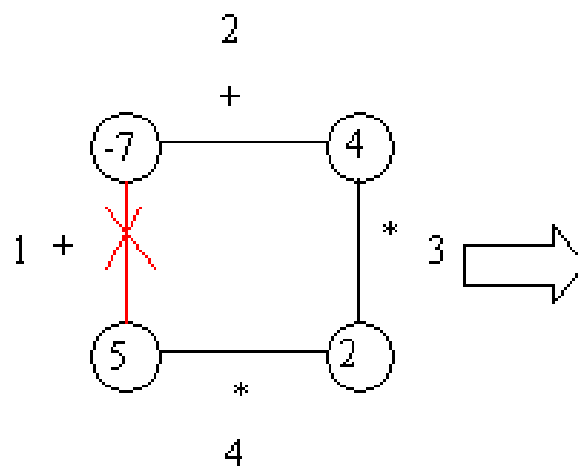
- ✧ 多边形是一个单人玩的游戏，开始时有一个N个顶点的多边形。如图，这里 $N=4$ 。
- ✧ 每个顶点有一个整数标记，每条边上有一个“+”号或“*”号。边从1编号到N。
- ✧ 第一步，一条边被拿走；随后各步包括如下：
- ✧ 选择一条边E和连接着E的两个顶点V1和 V2；
- ✧ 得到一个新的顶点，标记为V1与V2通过边E上的运算符的结果。
- ✧ 最后，游戏中没有边，游戏的得分为仅剩余的一个顶点的值。



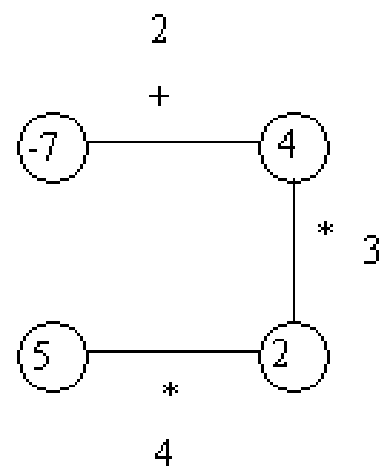
样例分析



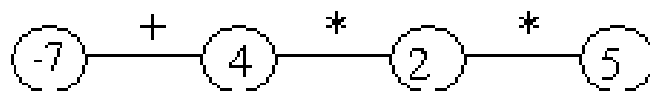
图一



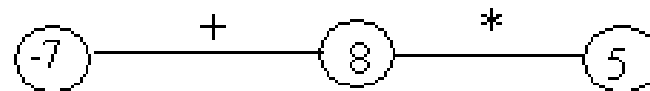
图二



图三



图四



图五



→ 分析

- ✧ 我们先枚举第一次删掉的边，然后再对每种状态进行动态规划求最大值。
- ✧ 用 $f(i, j)$ 表示从点 i 到点 j 进行删边操作所能得到的最大值
- ✧ $num(i)$ 表示第 i 个顶点上的数，若为加法，那么：

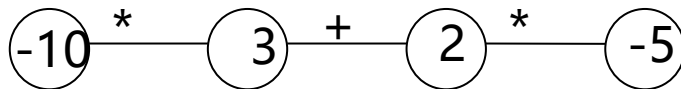
$$f(i, j) = \max \sum_{k=1}^{i-1} (f(k, i) + f(j - k, i + k))$$

$$f(i, i) = num(i)$$



进一步分析

- 最后，我们允许顶点上出现负数。以前的方程还适不适用呢？



图六

- 这个例子的最优解应该是 $(3+2)*(-10)*(-5)=250$ ，然而如果沿用以前的方程，得出的解将是 $((-10)*3+2)*(-5)=125$
- 为什么？我们发现，两个负数的积为正数；这两个负数越小，它们的积越大。我们从前的方程，只是尽量使得局部解最大，而从来没有想过负数的积为正数这个问题。



→ 分析

✧ 对于加法，两个最优相加肯定最优，而对于乘法

✧ 求最大值：

✧ 正数 $\times$ 正数，如果两个都是最大值，则结果最大

✧ 正数 $\times$ 负数，正数最小，负数最大，则结果最大

✧ 负数 $\times$ 负数，如果两个都是最小值，则结果最大

✧ 求最小值：

✧ 正数 $\times$ 正数，如果两个都是最小值，则结果最小

✧ 正数 $\times$ 负数，正数最大，负数最小，则结果最小

✧ 负数 $\times$ 负数，如果两个都是最大值，则结果最小



→ 最终?

- ✧ 我们引入函数fmin和fmax来解决这个问题。
fmax(i,j) 表示从点i开始，到点j为止进行删边操作所能得到的最大值，fmin(i,j) 表示最小值。
- ✧ 当OP= '+'

$$Fmax(i,j)=\max\{fmax(i,k)+fmax(k+1,j)\}$$

$$Fmin(i,j)=\min\{fmin(i,k)+fmin(k+1,j)\}$$



→ 分析

当OP='*'

$$f_{\max}(i, j) = \begin{cases} f_{\max}(i, k) * f_{\max}(k+1, j) \\ f_{\max}(i, k) * f_{\min}(k+1, j) \\ f_{\min}(i, k) * f_{\max}(k+1, j) \\ f_{\min}(i, k) * f_{\min}(k+1, j) \end{cases}$$

$$f_{\min}(i, j) = \begin{cases} f_{\min}(i, k) * f_{\min}(k+1, j) \\ f_{\max}(i, k) * f_{\min}(k+1, j) \\ f_{\min}(i, k) * f_{\max}(k+1, j) \\ f_{\max}(i, k) * f_{\max}(k+1, j) \end{cases}$$



完美解决

✧ 初始值

$$F_{\max}(i,i) = \text{num}(i)$$

$$F_{\min}(i,i) = \text{num}(i)$$

$$1 \leq i \leq k \leq j \leq n$$

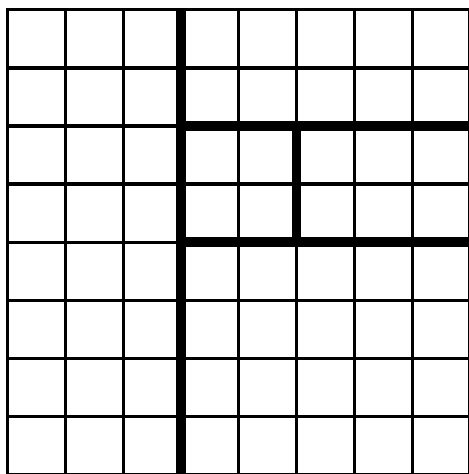
✧ 空间复杂度: $O(n^2)$

✧ 时间复杂度: 先要枚举每一条边为 $O(n)$, 然后动态规划为 $O(n^3)$, 因此总为 $O(n^4)$ 。

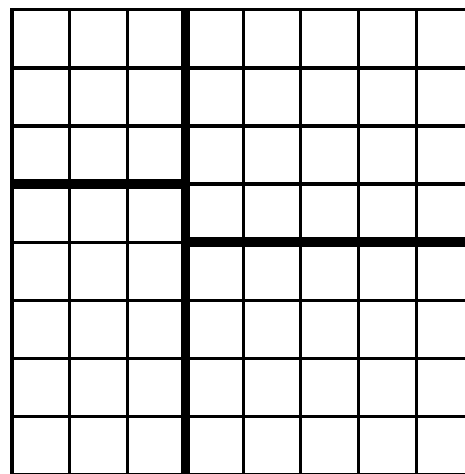


棋盘分割

- 将一个 8×8 的棋盘进行如下分割：将原棋盘割下一块矩形棋盘并使剩下部分也是矩形，再将剩下的部分继续如此分割，这样割了 $(n-1)$ 次后，连同最后剩下的矩形棋盘共有 n 块矩形棋盘。（每次切割都只能沿着棋盘格子的边进行）



允许的分割方案



不允许的分割方案



任务：

棋盘上每一格有一个分值，一块矩形棋盘的总分为其所含各格分值之和。现在需要把棋盘按上述规则分割成n块矩形棋盘，并使各矩形棋盘总分的均方差最小。

- 均方差：

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

- 算术平均值：

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$



样例

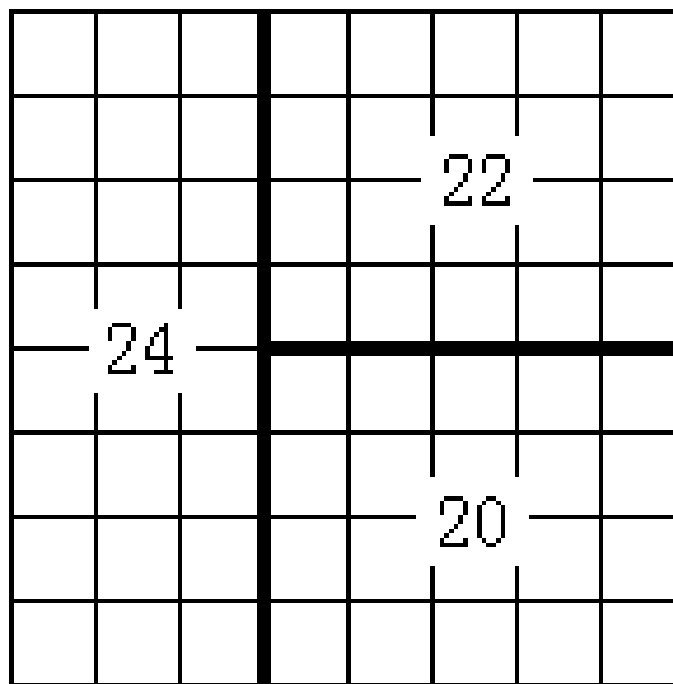
输入

3

```
1 1 1 1 1 1 1 3
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 0
1 1 1 1 1 1 0 3
```

输出

1.633



→ 均方差公式化简

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

$$\sigma^2 = \sum_{i=1}^n (x_i - \bar{x})^2 / n = \sum_{i=1}^n (x_i^2 - 2x_i \bar{x} + \bar{x}^2) / n$$

$$= (n\bar{x}^2 + \sum_{i=1}^n x_i^2 - 2\bar{x} \sum_{i=1}^n x_i) / n = \bar{x}^2 + \frac{1}{n} \sum_{i=1}^n x_i^2 - \frac{2\bar{x}}{n} \sum_{i=1}^n x_i$$

$$= \bar{x}^2 + \frac{1}{n} \sum_{i=1}^n x_i^2 - 2\bar{x}^2 = \frac{1}{n} \sum_{i=1}^n x_i^2 - \bar{x}^2$$



→ 分析

- 由化简后的均方差公式：

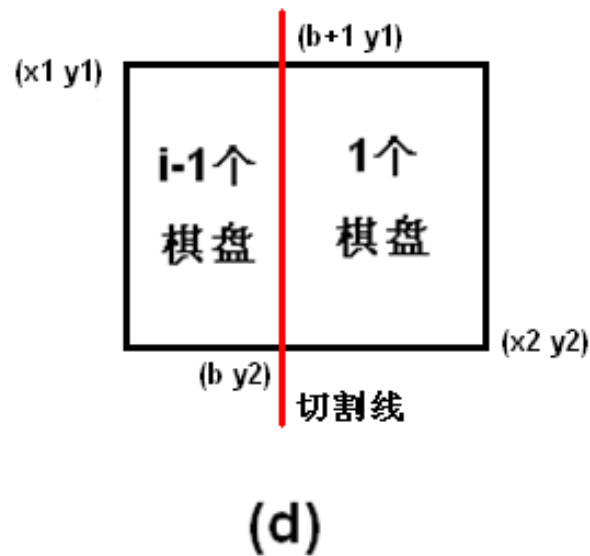
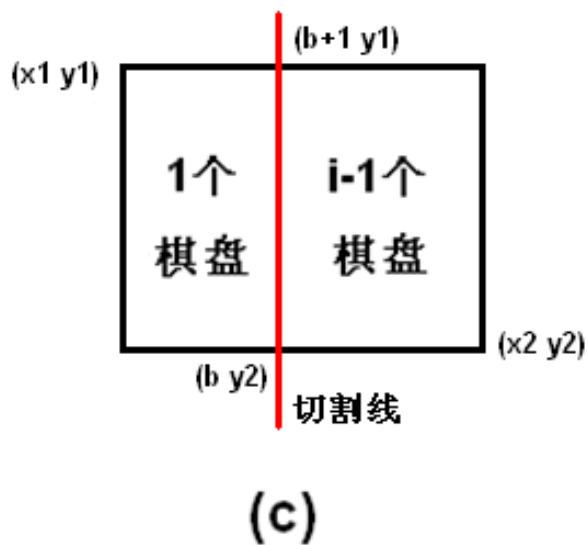
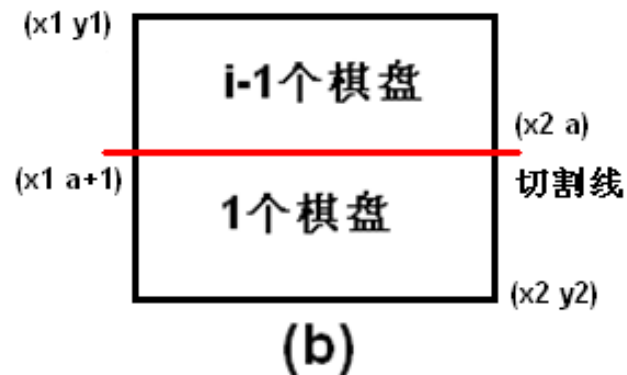
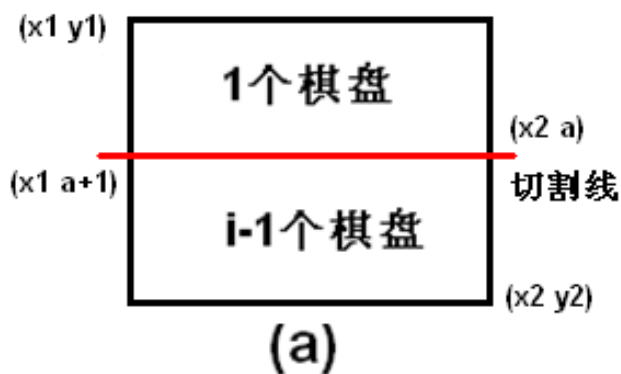
$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n x_i^2 - \bar{x}^2$$

- 可知，均方差的平方为每格数的平方和除以n，然后减去平均值的平方，而后者是一个已知数。
- 因此，在棋盘切割的各种方案中，只需使得每个棋盘内各数值的平方和最小即可。
- 因此，我们需要求出各棋盘分割后的每个棋盘各数平方和的最小值，设为w，那么
- 答案为：

$$ans = \sqrt{w/n - \bar{x}^2}$$



棋盘切割后的四种情况





→ 动态规划

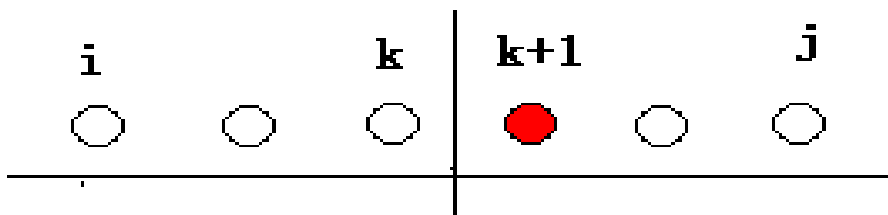
- 设 $F(i, x_1, y_1, x_2, y_2)$ 表示以 $[x_1, y_1][x_2, y_2]$ 为四边形对角线的棋盘切割成 k 块的各块数值总平方和的最小值，则有：

$$f(i, x_1, y_1, x_2, y_2) = \min \begin{cases} f(i-1, x_1, a+1, x_2, y_2) + D[x_1, y_1][x_2, a], \text{横切, 上面不动} \\ f(i-1, x_1, y_1, x_2, a) + D[x_1, a+1][x_2, y_2], \text{横切, 下面不动} \\ f(i-1, b+1, y_1, x_2, y_2) + D[x_1, y_1][b, y_2], \text{竖切, 左面不动} \\ f(i-1, x_1, y_1, b, y_2) + D[b+1, y_1][x_2, y_2], \text{竖切, 右面不动} \end{cases}$$
- $1 \leq x_1, x_2, x_3, x_4 \leq 8, 1 \leq i \leq n$ 。
- 设棋盘边长为 m , 则状态数为 nm^4 , 决策数最多 m 。
- 先预处理从左上角 $(1, 1)$ 到右下角 (i, j) 的棋盘和时间复杂度为 $O(m^2)$, 因此转移为 $O(1)$, 总时间复杂度为 $O(nm^5)$ 。



→ 总结

- ✧ 该类问题的基本特征是能将问题分解成为两两合并的形式。解决方法是对整个问题设最优值，枚举合并点，将问题分解成为左右两个部分，最后将左右两个部分的最优值进行合并得到原问题的最优值。有点类似分治的解题思想。
- ✧ 设前*i*到*j*的最优值，枚举剖分（合并）点，将(*i,j*)分成左右两区间，分别求左右两边最优值，如下图。



- ✧ 状态转移方程的一般形式如下：
- ✧ $F(i,j) = \text{Max}\{F(i,k) + F(k+1,j) + \text{决策}, k \text{ 为划分点}\}$



→ 游艇租用问题

✧ 问题描述:

长江俱乐部在长江设置了 n 个游艇出租站 $1, 2, \dots, n$ ，游客可在这些游艇出租站租用游艇，并在下游的任何一个游艇出租站归还游艇。游艇出租站 i 到游艇出租站 j 之间的租金为 $r(i, j)$ ，设计一个算法，计算出从出租站1到出租站 n 所需要的最少租金。

✧ 样例输入:

3

5 15

7

样例输出:

12



➔ 删数问题

➤ 问题描述

现有 n 个正整数组成的序列 a ，从中删除一个数，得分是其本身同左、右相邻的数的乘积，然后再在剩余的整数中继续删除，注意序列两端的数字 a_1 和 a_n 是不能删除的，求这样删除 $n-2$ 个整数后的最大得分。例如有四个数3、4、5、6，按照先4后5的删除顺序，其得分为 $345+356=150$ ，按照先5后4的删除顺序，其得分为 $456+346=192$ ，因此最大得分为192。



→ 最大的算式

➤ 问题描述

题目很简单，给出N个数字，不改变它们的相对位置，在中间加入K个乘号和N-K-1个加号，（括号随便加）使最终结果尽量大。

因为乘号和加号一共就是N-1个了，所以恰好每两个相邻数字之间都有一个符号。例如：

N=5，K=2，5个数字分别为1、2、3、4、5，可以加成：

$$1 * 2 * (3+4+5)=24$$

$$1 * (2+3) * (4+5)=45$$

$$(1+2+3) * (4+5)=45$$

.....



练习题目

✧ P2858

✧ P3146

✧ P3147

The background features a central point from which several wide, colorful rays (in shades of blue, orange, red, and grey) radiate outwards. Faint, light green binary code (0s and 1s) is scattered across the background, particularly on the right side. A small, stylized line graph with orange and blue lines is also visible on the right side.

Thank you!